Derek Bellin
CISC 106

About the URL of the Website

https://derekb-udel.github.io/Accuracy-Shot-Site/—-about

https://derekb-udel.github.io/Accuracy-Shot-Site/

LLM Documentation

User: I need to create a Basketball Shot Predictor website for my programming project. I want it to use physics to predict whether a basketball shot will go in. It should have inputs for shot angle, starting height, ball speed, and distance from the hoop. I also want it to graph the ball's path, show whether the shot goes in, and include shot history and shooting statistics. I want to use Python, NumPy, Matplotlib, and Drafter. Can you create the code and include functions, dataclasses, input validation, and tests?

ChatGPT: I can organize the project into separate functions and pages. I'll use projectile motion calculations to predict the shot, Matplotlib to graph the trajectory, and Drafter to build the website. I'll also include a way to save previous shots and calculate shooting percentages.

"""Basketball Shot Predictor.

You type in how you throw the ball (angle, speed, height) and where the hoop
is. The program uses physics to guess if the shot goes in, then draws the
path of the ball on a graph.
"""
import math
from dataclasses import dataclass

import matplotlib

```
matplotlib.use("Agg")  # ADVANCED: draw graphs without opening a pop-up window (GitHub has no screen)
import matplotlib.pyplot as plt  # noqa: E402  (has to come after matplotlib.use)
import numpy as np  # noqa: E402
from drafter import (Button, CheckBox, Header, MatPlotLibPlot, Page,  # noqa: E402
                     SelectBox, Table, TextBox, assert_equal, route,
                     set_site_information, start_server)

#
# STEP 1: Constants (numbers that never change, so they are allowed)
#
GRAVITY = 9.80665         # how fast things fall, in m/s^2
HOOP_HEIGHT = 3.05        # height of a real basketball rim, in meters
HOOP_TOLERANCE = 0.20     # how close to the hoop still counts as a swish, in meters
MAX_DEFENDER_HEIGHT = 1e100    # not a real limit, just stops the numbers getting too big for Python
MAX_DEFENDER_DISTANCE = 1e100  # same idea: the defender can stand as far away as you like
MAX_DISTANCE = 1_000_000   # farthest hoop allowed (1000 km, so basically anywhere)
MAX_SPEED = 1e100         # not a real limit, just stops the numbers getting too big for Python
CUSTOM_CHOICE = "Custom"
DISTANCE_OPTIONS = {      # name in the dropdown -> distance in meters
    "Free throw (4.6 m)": 4.6,
    "Mid-range (6.0 m)": 6.0,
    "Three-pointer (7.2 m)": 7.2,
    "Half court (14.0 m)": 14.0,
}


#
# STEP 2: Dataclasses (the "things" the program keeps track of)
#
@dataclass
class ShotResult:
    """One shot that was already taken: what happened and how close it was."""
    outcome: str          # "Swish!", "Short", "Long", or "Blocked"
    peak_height: float    # how high the ball got, in meters
    distance_from: float  # how far from the hoop it ended up, in meters (0.0 = scored)
```

```
@dataclass
class State:
    """Everything the website remembers while you move between pages."""
    current_angle: float          # launch angle, in degrees
    current_speed: float          # ball speed, in meters per second
    current_height: float         # height the ball starts at, in meters
    current_distance: float       # how far away the hoop is, in meters
    defender_on: bool             # is the defender standing there?
    defender_height: float        # how tall the defender is, in meters
    defender_distance: float      # how far the defender stands from you, in meters
    shot_history: list[ShotResult]  # every shot taken so far
    message: str                  # note shown at the top of the first page


#
# STEP 3: Helper functions (small jobs that the pages use)

def parse_number(text: str) -> float:
    """Turn typed text into a number, or give back -1.0 if it isn't a real number.

    ADVANCED TECHNIQUE: try/except lets us try something that might fail.
    float("abc") causes an error, and without try/except the website would
    crash. Instead we catch the error and return -1.0, which the checks
    later treat as bad input. math.isfinite also rejects "nan" and "inf",
    which float() would otherwise accept even though they aren't useful.
    """
    try:
        number = float(text)
    except ValueError:
        return -1.0
    if not math.isfinite(number):
        return -1.0
    return number


def get_distance(choice: str, custom_text: str) -> float:
    """Get the hoop distance from the dropdown, or from the custom box."""
    if choice == CUSTOM_CHOICE:
        return parse_number(custom_text)
```

```
    return DISTANCE_OPTIONS[choice]


def distance_label(distance: float) -> str:
    """Find the dropdown name for a distance, or "Custom" if none match."""
    for label, meters in DISTANCE_OPTIONS.items():
        if meters == distance:
            return label
    return CUSTOM_CHOICE


def peak_height(angle: float, speed: float, height: float) -> float:
    """Work out how high the ball gets at the top of its arc, in meters."""
    vertical_speed = speed * np.sin(np.radians(angle))
    # Physics: height gained going up = (upward speed squared) / (2 * gravity)
    return float(height + vertical_speed ** 2 / (2 * GRAVITY))


def ball_height_at(x: float, angle: float, speed: float, height: float) -> float:
    """Work out how high the ball is when it is x meters away from you."""
    horizontal_speed = speed * np.cos(np.radians(angle))
    vertical_speed = speed * np.sin(np.radians(angle))
    time = x / horizontal_speed
    return float(height + vertical_speed * time - 0.5 * GRAVITY * time ** 2)


def time_at_hoop(angle: float, speed: float, height: float) -> float:
    """Work out when the ball drops down to hoop height, or -1.0 if it never does.

    The ball can pass hoop height twice: once going up and once coming down.
    The quadratic formula gives both times, and we use the later one because
    the ball has to be coming down to go in.
    """
    vertical_speed = speed * np.sin(np.radians(angle))
    discriminant = vertical_speed ** 2 - 2 * GRAVITY * (HOOP_HEIGHT - height)
    if discriminant < 0:  # negative means the ball never gets as high as the hoop
        return -1.0
    return float((vertical_speed + np.sqrt(discriminant)) / GRAVITY)
```

```
def miss_amount(angle: float, speed: float, height: float,
                distance: float) -> float:
    """Work out how far past the hoop the ball lands (negative means it fell short)."""
    time = time_at_hoop(angle, speed, height)
    if time < 0:
        return -distance  # the ball never reached the rim at all
    landing_x = speed * np.cos(np.radians(angle)) * time
    return float(landing_x - distance)


def defender_blocks(angle: float, speed: float, height: float, distance: float,
                    defender_distance: float, defender_height: float) -> bool:
    """Say whether the ball hits the defender on the way to the hoop."""
    if defender_distance >= distance:  # the defender is past the hoop, so he can't block
        return False
    ball_height = ball_height_at(defender_distance, angle, speed, height)
    # The ball has to still be in the air (not below 0) and not above his head
    return 0 <= ball_height <= defender_height


def judge_shot(angle: float, speed: float, height: float, distance: float,
               defender_on: bool, defender_height: float,
               defender_distance: float) -> str:
    """Decide what happens to the shot: "Blocked", "Short", "Long", or "Swish!"."""
    # IF 1: Does the ball hit the defender?
    if defender_on and defender_blocks(angle, speed, height, distance,
                                       defender_distance, defender_height):
        return "Blocked"
    # IF 2: Does the ball never even get as high as the hoop?
    if time_at_hoop(angle, speed, height) < 0:
        return "Short"
    miss = miss_amount(angle, speed, height, distance)
    # IF 3: Did it land on the hoop, before it, or past it?
    if abs(miss) <= HOOP_TOLERANCE:
        return "Swish!"
    if miss < 0:
        return "Short"
    return "Long"
```

```
def trajectory_points(angle: float, speed: float,
                      height: float) -> tuple[np.ndarray, np.ndarray]:
    """Make lists of points (as NumPy arrays) that trace the ball's whole flight."""
    vx = speed * np.cos(np.radians(angle))
    vy = speed * np.sin(np.radians(angle))
    # How long until the ball hits the floor
    ground_time = (vy + np.sqrt(vy ** 2 + 2 * GRAVITY * height)) / GRAVITY
    times = np.linspace(0, ground_time, 100)  # 100 moments spread evenly across the
flight
    return vx * times, height + vy * times - 0.5 * GRAVITY * times ** 2


def validate_inputs(angle: float, speed: float, height: float,
                    distance: float, defender_height: float,
                    defender_distance: float) -> str:
    """Check the typed numbers. Give back an error message, or "" if all is fine."""
    if not 1 <= angle <= 89:
        return "Angle must be a number from 1 to 89 degrees."
    if not 0 < speed <= MAX_SPEED:
        return "Speed must be a number above 0."
    if not 0 < height <= 3:
        return "Starting height must be a number between 0 and 3 m."
    if not 0 < distance <= MAX_DISTANCE:
        return f"Distance must be a number above 0 (up to {MAX_DISTANCE} m)."
    if not 0 < defender_height <= MAX_DEFENDER_HEIGHT:
        return "Defender height must be a number above 0."
    if not 0 < defender_distance <= MAX_DEFENDER_DISTANCE:
        return "Defender distance must be a number above 0."
    return ""


# STEP 4: Routes (each one is a page, or a button's action)
—
@route
def index(state: State) -> Page:
    """First page: where you type in your shot."""
    return Page(state, [
        Header("Basketball Shot Predictor"),
        "Enter your shot and see if it goes in!",
        state.message,
```

```
        "Launch angle (1-89 degrees):",
        TextBox("angle_text", str(state.current_angle)),
        "Ball speed (m/s, no limit):",
        TextBox("speed_text", str(state.current_speed)),
        "Starting height (m):",
        TextBox("height_text", str(state.current_height)),
        "Distance from hoop:",
        SelectBox("distance_choice",
                list(DISTANCE_OPTIONS) + [CUSTOM_CHOICE],
                distance_label(state.current_distance)),
        "Custom distance in meters (only used if 'Custom' is chosen):",
        TextBox("custom_text", str(state.current_distance)),
        CheckBox("defender", state.defender_on),
        "Add a defender",
        "Defender height (m):",
        TextBox("defender_height_text", str(state.defender_height)),
        "Defender distance from you (m):",
        TextBox("defender_distance_text", str(state.defender_distance)),
        Button("Fire Shot", fire_shot),
        Button("Shot History", show_history),
        Button("Stats", show_stats),
    ])


@route
def fire_shot(state: State, angle_text: str, speed_text: str,
              height_text: str, distance_choice: str, custom_text: str,
              defender: bool, defender_height_text: str,
              defender_distance_text: str) -> Page:
    """Take what was typed, check it, judge the shot, and save the result."""
    angle = parse_number(angle_text)
    speed = parse_number(speed_text)
    height = parse_number(height_text)
    distance = get_distance(distance_choice, custom_text)
    defender_height = parse_number(defender_height_text)
    defender_distance = parse_number(defender_distance_text)

    error = validate_inputs(angle, speed, height, distance,
                            defender_height, defender_distance)
    if error != "":
```

```
    state.message = error
    return index(state)

  # Save what the user typed into the state so the pages remember it
  state.current_angle = angle
  state.current_speed = speed
  state.current_height = height
  state.current_distance = distance
  state.defender_on = defender
  state.defender_height = defender_height
  state.defender_distance = defender_distance
  state.message = ""

  outcome = judge_shot(angle, speed, height, distance, defender,
                       defender_height, defender_distance)
  distance_from = 0.0
  if outcome != "Swish!":
    distance_from = round(abs(miss_amount(angle, speed, height, distance)), 2)
  state.shot_history.append(
    ShotResult(outcome, round(peak_height(angle, speed, height), 2),
               distance_from))
  return show_result(state)


@route
def show_result(state: State) -> Page:
  """Result page: shows what happened on the latest shot."""
  latest = state.shot_history[-1]
  return Page(state, [
    Header("Result: " + latest.outcome),
    f"Peak height: {latest.peak_height} m",
    f"Missed the hoop by: {latest.distance_from} m",
    Button("See Graph", show_graph),
    Button("Shoot Again", index),
  ])


@route
def show_graph(state: State) -> Page:
  """Graph page: draws the path of the latest shot."""
```

```
    x_values, y_values = trajectory_points(
        state.current_angle, state.current_speed, state.current_height)
    plt.figure()
    plt.plot(x_values, y_values, label="Ball path")
    plt.scatter([state.current_distance], [HOOP_HEIGHT],
                color="red", label="Hoop")
    right_edge = max(float(x_values.max()), state.current_distance)
    if state.defender_on:
        plt.plot([state.defender_distance, state.defender_distance],
                 [0, state.defender_height],
                 color="black", linewidth=4, label="Defender")
        right_edge = max(right_edge, state.defender_distance)
    plt.ylim(bottom=0)
    plt.xlim(0, right_edge * 1.05)  # stretch the graph so the hoop and defender fit
    plt.xlabel("Distance from shooter (m)")
    plt.ylabel("Height (m)")
    plt.title("Shot Trajectory")
    plt.legend()
    return Page(state, [
        Header("Shot Trajectory"),
        MatPlotLibPlot(),
        Button("Back to Simulator", index),
    ])


@route
def show_history(state: State) -> Page:
    """History page: a table of every shot taken."""
    rows = [["Shot #", "Outcome", "Peak height (m)", "Missed by (m)"]]
    # LOOP 1: add one table row for each past shot
    for number, shot in enumerate(state.shot_history, start=1):
        rows.append([str(number), shot.outcome, str(shot.peak_height),
                     str(shot.distance_from)])
    return Page(state, [
        Header("Shot History"),
        Table(rows),
        Button("Clear History", reset_history),
        Button("Back to Simulator", index),
    ])
```

```
@route
def show_stats(state: State) -> Page:
    """Stats page: how many shots went in and the average peak height."""
    made = 0
    total_peak = 0.0
    # LOOP 2: count the shots that went in and add up all the peak heights
    for shot in state.shot_history:
        if shot.outcome == "Swish!":
            made += 1
        total_peak += shot.peak_height
    attempts = len(state.shot_history)
    percentage = 0.0
    average_peak = 0.0
    if attempts > 0:  # can't divide by zero if no shots were taken
        percentage = round(made / attempts * 100, 1)
        average_peak = round(total_peak / attempts, 2)
    return Page(state, [
        Header("Stats & Analytics"),
        f"Shots taken: {attempts}",
        f"Shots made: {made}",
        f"Shooting percentage: {percentage}%",
        f"Average peak height: {average_peak} m",
        Button("Back to Simulator", index),
    ])


@route
def reset_history(state: State) -> Page:
    """Clear the list of past shots."""
    state.shot_history = []
    state.message = "History cleared."
    return index(state)

# STEP 5: Unit tests (these run when the program starts to check our work

def make_test_state() -> State:
    """Make a new State to use in the tests."""
    return State(52.0, 7.6, 2.0, 4.6, False, 2.1, 1.5, [], "")
```

```
assert_equal(distance_label(123.0), CUSTOM_CHOICE)
assert_equal(round(peak_height(90, 9.80665, 2.0), 2), 6.9)
assert_equal(time_at_hoop(10, 5, 2.0), -1.0)  # too flat and slow to reach the hoop
assert_equal(judge_shot(52, 7.6, 2.0, 4.6, False, 2.1, 1.5), "Swish!")
assert_equal(judge_shot(52, 6.0, 2.0, 4.6, False, 2.1, 1.5), "Short")
assert_equal(judge_shot(52, 9.0, 2.0, 4.6, False, 2.1, 1.5), "Long")
# A flat 5 degree shot is about 2 m high when it reaches the defender.
# A 2.1 m defender blocks it, a 0.5 m defender doesn't, and a defender
# standing past the hoop doesn't either.
assert_equal(judge_shot(5, 8.0, 2.0, 4.6, True, 2.1, 1.5), "Blocked")


# Route tests: a route gives back a Page, and page.state holds the changes
fired = fire_shot(make_test_state(), "52", "7.6", "2.0",
                  "Free throw (4.6 m)", "", False, "2.1", "1.5")
assert_equal(len(fired.state.shot_history), 1)
assert_equal(fired.state.shot_history[0].outcome, "Swish!")
tall = fire_shot(make_test_state(), "52", "7.6", "2.0",
                 "Free throw (4.6 m)", "", True, "3.0", "2.5")
assert_equal(tall.state.defender_height, 3.0)
assert_equal(tall.state.defender_distance, 2.5)
far = fire_shot(make_test_state(), "52", "7.6", "2.0",
                CUSTOM_CHOICE, "250", False, "2.1", "1.5")
assert_equal(far.state.current_distance, 250.0)
fast = fire_shot(make_test_state(), "52", "5000", "2.0",
                 "Free throw (4.6 m)", "", False, "2.1", "1.5")
assert_equal(fast.state.current_speed, 5000.0)
assert_equal(reset_history(fired.state).state.shot_history, [])
assert_equal(show_history(make_test_state()).state.shot_history, [])
assert_equal(show_stats(make_test_state()).state.shot_history, [])
# index, show_result, and show_graph only show things on screen (and
# show_graph draws a picture), so they can't be unit tested. We check them
# by clicking through the website.


start_server(State(52.0, 7.6, 2.0, 4.6, False, 2.1, 1.5, [], ""))
```

User: This looks good. Can you make sure the website is easy to use and that the results clearly show whether the shot went in, was short, or went long?

ChatGPT: Yes. I'll make the results easier to understand and organize the website so users can enter their shot information, view the graph, and check their statistics.